

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I JAYADEEP.J(192511304) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.
Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWER SHEET

Artificial Intelligence (CSA17) – CO1

Question 1: Intelligent Agents in a Smart Home System

An agent is anything that perceives its environment through sensors and acts on it through actuators. In the smart home, the sensors are motion detectors, temperature sensors, door/window contacts, cameras and the clock; the actuators are the lights, the heating and cooling system, locks and the alarm. The four standard agent types differ in how much they remember and how they decide.

Simple reflex agent

This type acts only on the current percept using condition–action rules, with no memory and no model of the world. In the smart home it would work like: if motion is detected and it is dark, then switch on the lights; if the window is open and the alarm is armed, then sound the alarm; if the temperature is above 28 °C, then switch on the air conditioner. It is fast, simple and reliable, which is good for safety triggers. The weakness is that it only works when the correct decision can be made from the current percept alone. It cannot tell whether the room has been empty for an hour, so it may leave the lights on, or it may turn them off while someone is sitting still and not triggering the motion sensor.

Model-based reflex agent

This agent keeps an internal state describing the parts of the world it cannot see right now, and updates it using a model of how the world changes and how its own actions affect it. For the smart home, it would remember that someone entered the living room two minutes ago and has not left, so the lights should stay on even when the motion sensor is quiet. It could track which rooms are occupied, whether the house is empty, and how fast a room is cooling. This fixes the partial observability problem of the reflex agent, but it still just reacts to the situation and has no sense of what it is trying to achieve.

Goal-based agent

A goal-based agent chooses actions by considering whether they will bring it closer to a goal, which may require search or planning. Goals could be "keep the bedroom at 22 °C by 10 pm" or "have the house secured by midnight". The agent can plan ahead, for example starting the heater early so the room is warm by bedtime, or locking every door in a sequence. It is more flexible than a rule table because changing the goal changes the behaviour without rewriting rules. The limitation is that goals are all-or-nothing: it knows reaching 22 °C is "good" but cannot say which of two ways of reaching it is better.

Utility-based agent

A utility-based agent uses a utility function that scores how desirable a state is, and picks the action with the highest expected utility. This lets it balance competing aims, such as comfort against energy cost, or security against convenience. In the smart home, it can decide that dropping the

temperature by one degree saves a significant amount of energy for very little loss of comfort, or that pre-cooling at off-peak tariff hours is better than cooling at peak. Utility also handles uncertainty, since the agent can weigh the probability of someone returning soon against the cost of heating an empty house. With learning added, the utility function can be tuned to the user's preferences over time, for example noticing that the user always lowers the lights after 9 pm.

Which agent is best?

No single type covers all the requirements, so I would recommend a hybrid, layered design:

- **Reflex layer for security and safety.** Immediate responses to events like forced entry, smoke or a gas leak need to be quick and predictable, and shouldn't depend on any planning or learning.
- **Model-based layer for awareness.** Maintains occupancy, time-of-day patterns and room temperature trends so that comfort decisions are not based on a single sensor reading.
- **Goal-based layer for planning.** Sets goals such as a target temperature at a given time and plans the heating, cooling and lighting actions to reach them.
- **Utility-based layer for trade-offs.** Chooses between options by combining comfort, energy cost and security into a single score, which is what gives the energy efficiency.
- **Learning element.** Observes the user's overrides and habits and adjusts the preferences and utility weights, which is the "learning user preferences over time" requirement.

Comfort comes from the model-based, goal-based and learning parts; energy efficiency mainly from the utility-based part; and security from the reflex layer, with the model-based layer helping to spot unusual patterns such as movement when the house is meant to be empty. A purely reflex or purely goal-based agent would be fine for one of these needs but poor on the others. The hybrid costs more in computation and design effort, but the home has several different needs that justify it.

Question 2: Robot in an Unknown Maze

The robot starts at some cell, knows nothing about the layout, and has to reach the exit. Because it does not know where the exit is, it cannot use a heuristic that estimates the distance to the goal, and this is why uninformed (blind) search strategies are used. The robot treats each junction as a node, and the corridors as the edges that connect them.

Uniform Cost Search (UCS)

UCS always expands the node with the lowest path cost $g(n)$ so far, using a priority queue. If every step costs 1 (a plain maze), it behaves like breadth-first search, exploring the maze in rings of increasing distance. If corridors have different costs, such as a muddy section that takes longer, UCS accounts for it and finds the cheapest route, not just the shortest. It is complete provided each step costs at least some positive ϵ , and it is optimal.

- **Time:** $O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$, where b is the branching factor, C^* is the cost of the optimal solution and ϵ is the minimum step cost. It may explore a lot of cheap nodes before reaching the exit.
- **Space:** also $O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$, because the whole frontier has to be kept in memory.
- **Advantage:** guarantees the least-cost path.
- **Disadvantage:** heavy memory use, and it explores in all directions, including those that lead away from the exit.

Iterative Deepening Search (IDS)

IDS runs depth-limited depth-first search repeatedly with limits 0, 1, 2, 3 and so on until the exit is found. It joins the low memory use of depth-first search with the completeness of breadth-first search. At first it sounds wasteful, because nodes at shallow levels are generated again and again, but most nodes in a tree are at the deepest level, so the repeated work is small. For example, with $b = 10$ and $d = 5$, IDS generates around 123,450 nodes against 111,110 for BFS, only about 11% more.

- **Time:** $O(b^d)$, where d is the depth of the shallowest exit.
- **Space:** $O(b \cdot d)$, which is linear in depth, much smaller than BFS or UCS.
- **Advantage:** complete (for finite branching), optimal when all step costs are equal, and uses little memory, so it is suitable for large mazes.
- **Disadvantage:** repeats work at shallow depths, and it is not cost-optimal when step costs differ.

	UCS	IDS
Complete	Yes (step cost $\geq \epsilon > 0$)	Yes (finite branching factor)
Optimal	Yes, least cost	Only if all step costs are equal
Time	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$	$O(b^d)$
Space	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$	$O(b \cdot d)$
Handles varying costs	Yes	No

Relating the problem to agent types and agent design

How the robot is designed decides what it can do with these algorithms:

- **Simple reflex robot:** a rule like "turn left if there is a wall ahead" or a wall-follower might work in some mazes, but it has no memory, so it can loop forever in a maze with a cycle. It cannot run UCS or IDS, since both depend on remembering explored nodes.
- **Model-based robot:** keeps a map and a set of visited cells, which are the necessary ingredients for search. Without them, the maze remains unobservable and any search would revisit cells endlessly.

- **Goal-based robot:** has "reach the exit" as an explicit goal, so it can use search algorithms to find a sequence of moves to it. This is the natural design for this problem.
- **Utility-based robot:** can weigh the cost of each corridor (time, battery, risk) and so benefits from UCS, which uses the cost information. It can also trade off a slightly longer but safer route.

So agent design affects decision-making directly: the more the agent remembers and the more it understands about goals and costs, the better the decisions it can make. One practical point is that in a truly unknown maze the robot has to physically move to discover the layout, and IDS's repeated re-exploration is cheap only when run on the map the robot has already stored in memory, not when it means physically retracing its steps.

Best combination

My choice is a **model-based, goal-based agent that uses IDS** when the corridors are of roughly equal length (a standard maze with unit step costs). IDS finds the shallowest exit, which is also the shortest path in that case, and it needs only $O(b \cdot d)$ memory, which matters for a small onboard robot. If the maze has corridors with clearly different costs, then a **utility-based, model-based agent with UCS** is the better pairing, since only UCS guarantees the least-cost path. In both cases, the model-based part is essential because the robot must remember the map it has built.

In summary, IDS is the best general choice for an unknown maze of uniform cost because of its low memory demand while still being complete and optimal, and UCS should be switched in when the costs vary and an optimal route matters more than memory.